

AgriMart — Multi-Branch Agriculture Shop Management

A full-stack billing and inventory management system for 4 shop branches. **Zero cloud cost** — runs 100% locally using SQLite.

Quick Start (5 minutes)

Prerequisites — Install These First

Tool	Download	Purpose
Python 3.10+	https://python.org/downloads	Backend server
Node.js 18+	https://nodejs.org	Frontend build
VS Code	https://code.visualstudio.com	Code editor

 **Windows tip:** When installing Python, check "Add Python to PATH"!

Step 1 — Open the Project in VS Code

File → Open Folder → select the `agrimart` folder

Open two terminals in VS Code (`Ctrl + ~`):

Step 2 — Start the Backend

In **Terminal 1**:

```
bash

cd backend

# Install Python packages (one time only)
pip install -r requirements.txt

# Start the backend server
python -m uvicorn main:app --reload --port 8000
```

You'll see: `Uvicorn running on http://127.0.0.1:8000`

The SQLite database (`agrimart.db`) is created automatically with all sample data.

Step 3 — Start the Frontend

In **Terminal 2**:

```
bash

cd frontend

# Install Node packages (one time only)
npm install

# Start the React app
npm run dev
```

You'll see: `Local: http://localhost:5173/`

Step 4 — Open the App

Visit: <http://localhost:5173>

Demo Login Credentials

User	Username	Password	Access
 Owner	<code>owner</code>	<code>admin123</code>	All shops + reports
 Shop 1 Staff	<code>arjun</code>	<code>shop1</code>	North branch only
 Shop 2 Staff	<code>meena</code>	<code>shop2</code>	South branch only
 Shop 3 Staff	<code>suresh</code>	<code>shop3</code>	East branch only
 Shop 4 Staff	<code>kavitha</code>	<code>shop4</code>	West branch only

Project Structure

```
agrimart/
├── backend/
│   ├── main.py          ← FastAPI app entry point
│   ├── database.py      ← SQLite schema + seed data
│   ├── auth.py          ← JWT authentication
│   ├── requirements.txt ← Python packages
│   ├── agrimart.db       ← SQLite database (auto-created)
│   └── routers/
│       ├── users.py      ← Login, user management
│       ├── shops.py      ← Shop summaries
│       ├── products.py   ← Product CRUD + stock
│       ├── bills.py      ← Billing + invoice
│       └── reports.py    ← Analytics
├── frontend/
│   ├── src/
│   │   ├── App.jsx       ← Router setup
│   │   └── pages/
│   │       ├── LoginPage.jsx
│   │       ├── Dashboard.jsx
│   │       ├── BillingPage.jsx
│   │       ├── InventoryPage.jsx
│   │       ├── BillHistoryPage.jsx
│   │       ├── ReportsPage.jsx
│   │       └── UsersPage.jsx
│   │   ├── components/
│   │   │   ├── Layout.jsx ← Top bar + bottom nav
│   │   │   └── UI.jsx     ← Shared components
│   │   └── context/
│   │       └── AuthContext.jsx
│   │   └── utils/
│   │       └── api.js      ← Axios + interceptors
│   ├── index.html
│   ├── vite.config.js
│   ├── tailwind.config.js
│   └── package.json
├── start-windows.bat    ← One-click Windows launch
└── start-mac-linux.sh   ← One-click Mac/Linux launch
```

🌟 Features

👑 Owner

- Dashboard with shop-wise KPIs and revenue bars
- Cross-shop bill history with filters
- Inventory with all-shop stock grid view
- Analytics: daily trend charts, top products, shop performance
- User management (add/activate/deactivate)
- Low stock alerts across all branches
- Printable invoices from any bill

👤 Salesperson

- Locked to their assigned shop only
 - Create bills with product search and cart
 - Auto stock deduction after each bill
 - Add / Edit products and update stock
 - View their shop's bill history
 - Print invoices
-

🔧 API Reference

The backend runs at `http://localhost:8000`

Method	Endpoint	Description
POST	/api/auth/login	Login → JWT token
GET	/api/products/?shop_id=1	List products with stock
POST	/api/products/	Create product
POST	/api/products/{id}/stock	Update stock level
POST	/api/bills/	Create bill (deducts stock)
GET	/api/bills/	List bills (filtered)
GET	/api/bills/{id}	Get full bill with items
GET	/api/reports/summary	Revenue summary
GET	/api/reports/products	Product-wise sales
GET	/api/reports/daily	Day-wise sales
GET	/api/reports/low-stock	Low stock alerts

Full interactive docs at: <http://localhost:8000/docs>

Configuration

Change the secret key (production)

In `backend/auth.py`:

```
python
SECRET_KEY = "your-secure-random-key-here"
```

Change low stock threshold

In `frontend/src/components/UI.jsx`:

```
js
```

```
// Change the number 10 to your preferred threshold
```

```
<LowStockBadge qty={qty} threshold={10} />
```

Add more shops

1. Insert into `shops` table in `database.py`
2. Add to `SHOPS_META` in `frontend/src/components/UI.jsx`

Database Backup

Your data is in `backend/agrimart.db` — a single file!

To back up:

```
bash
```

```
copy backend\agrimart.db backup\agrimart-backup.db # Windows
```

```
cp backend/agrimart.db backup/agrimart-backup.db # Mac/Linux
```

Future Upgrades

Feature	How to add
Multiple branches on network	Change <code>baseURL</code> in <code>api.js</code> to server IP
PostgreSQL	Replace <code>sqlite3</code> with <code>psycopg2</code> in <code>database.py</code>
SMS/WhatsApp invoices	Add Twilio or WATI API integration
Barcode scanner	Add <code>quagga2</code> npm package
Export to Excel	Add <code>openpyxl</code> to backend reports endpoint
Mobile app (PWA)	Already mobile-friendly! Add <code>manifest.json</code>

Troubleshooting

"pip not found" → Try `pip3` instead of `pip`

"Module not found" → Run `pip install -r requirements.txt` again

CORS error in browser → Make sure backend is running on port 8000

Port 8000 already in use → `python -m uvicorn main:app --port 8001` and update `vite.config.js` proxy target

Database locked → Close other connections, or delete `agrimart.db` to reset all data